What I actually did this week 5

So basically this week I focused on the **OpenAI side** of our project. Before I didnt really get why during my initial code was trying to hardcode the key on the file by mistake but now I kinda get it if you push it to GitHub by mistake anyone can use your key.

What I did was make sure we use **.env** for the secret stuff, specially:

- OPENAI_API_KEY – the main thing you need to call OpenAI
- OPENAI_MODEL – which model we use (I left default like gpt-4o-mini in example)
- OPENAI_MAX_TOKENS – so the answer doesnt become too long / too costly
- OPENAI_MAX_INPUT_CHARS – so if a file is HUGE we dont send whole thing at once and break the request

I also looked at services/openaiservices.js to understand
how generateSummary and answerQuery works. Its just sending a prompt to OpenAI and reading back text but not efficient right now but we can fix these our initial project works start functioning normally.

The app uses OpenAI in a few places:

1. When we summarize a file (process-code)
2. When we upload repo and it reads .js files and summarize each one
3. When we search the "whole codebase" and ask a question (/search)

So if OpenAI config is wrong, **everything** breaks or becomes slow.
Also I learned **.env.example** is just a template (safe to share), but **.env** is real secrets (should stay private)

What I understood / learned

- Env variables load when server starts if I change .env I must **restart** server (I forgot this once).
- Truncation means if input is too long, we cut it summary might look "half" but its becouse of safety limits.

Problems I faced

- I mixed up **template** vs **real env** sometimes.
- I didnt realise /search builds one giant string from Mongo – so costs can go up fast if DB has alot of files.

Conclusion

Week 5 for me was not about adding a new feature it was about doing **OpenAI properly** as a sknobs for model/tokens/input size, and understanding where OpenAI is called in our app. Next week I want to be more careful with /search becouse sending whole DB every time is ok for demo but not for real scale.

Files I mainly used / checked

- services/openaiservices.js
- .env
- controllers/codecontroller.js (shows the API flow)

Week 6

This week I didnt add a whole new API. I just made **uploadRepo** behave a bit smarter after we download a GitHub repo.

1) Save filePath when we store repo files

Before, when we saved each processed file to Mongo, we only saved stuff
like fileName, code, summary but our Code schema also has **filePath** and other endpoints (like when I upload files manually) already save filePath So for repo upload it felt inconsistant.
Now when I loop through processedFiles and create new Code({ ... }), I also set:

filePath: file.fileName

So basically I'm storing the **path/name inside the zip** as the "path" field. Its not a full Windows path on my laptop its more like the path GitHub gives inside the archive but atleast Mongo has something useful for later when we show "which file is this" in search or UI.

2) Delete the downloaded ZIP after we're done (

fetchRepo saves a ZIP somewhere under uploads/ (like uploads/repo.zip). If we never delete it, the folder fills up fast and wastes disk space (specially if I test many repos while learning).
So I added a finally block:

- I keep zipPath in a variable outside try so it still exists even if something errors.
- After try/catch, **finally always runs** (success or fail).
- If zipPath is set, I call fs.unlinkSync(zipPath) to delete the zip.
- If delete fails, I log it I dont want the whole server to crash just because delete failed.

Conclusion

Week 6 was a small but important cleanup week for **repo upload flow**: we **save filePath in Mongo** so data is more complete, and we **delete the ZIP after processing** so uploads/ doesnt become a junk folder. Next I might want to handle repos where default branch is not main (that could be a future week fix in githubService).